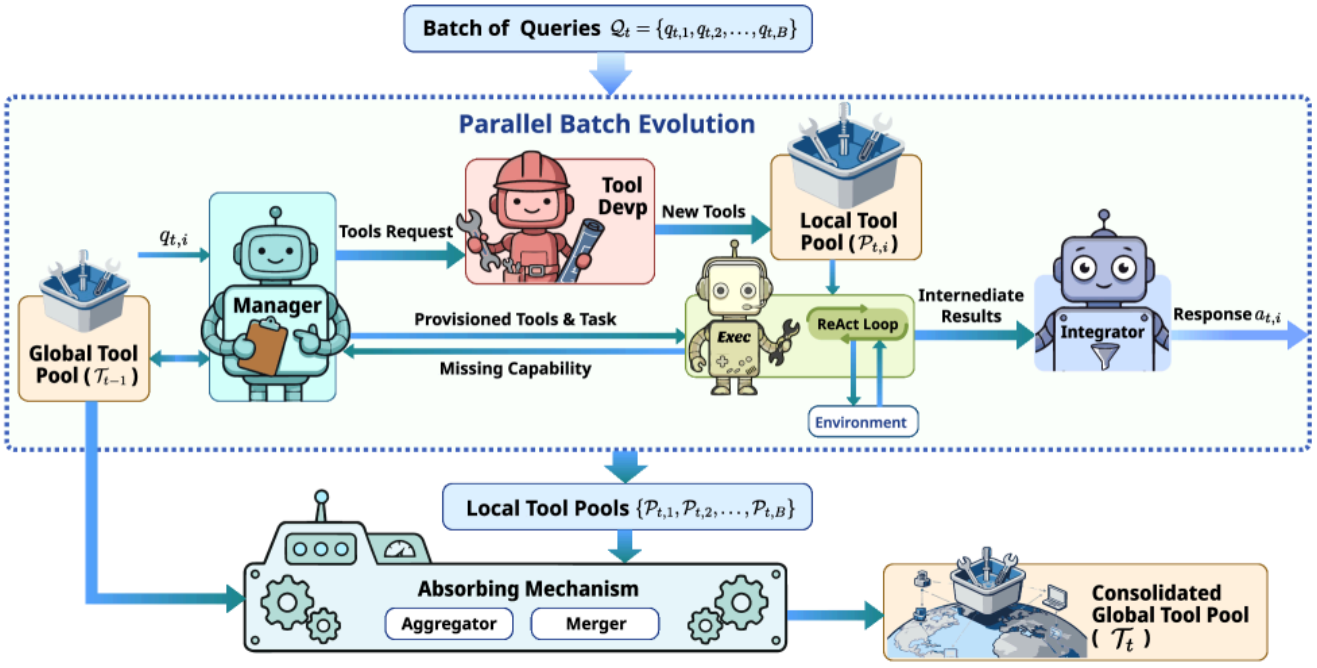
Yunjue Agent: In-Situ Self-Evolving Tool Synthesis and Convergence

> #Agents #Self-evolve #Tools



- What problem does this project solve?
  - Traditionally, the model cannot evolve itself: we need to train (finetune/alignment) it over time, which is time consuming, costly, and with high latency.
  - Workflow and context evolution may lack objectivity.
- What does this do?
  - We give a system of agents an empty tool repository at the start.
  - During the interaction with the users, the agents create tools themselves to populate the repository.
  - After a period of interaction, lower # of new tools will be produced -> the repo becomes matured.
- Details of Agents
  - Manager: Analyze the user's needs; find if the required tools exist; if not, guide the Tool Developer to create needed tools
  - Executor: Apply the tools; if there is a mistake or missing functionality, pause and notify the Manager.
  - Tool Developer: Create python-based tools and add the tool to the repository.
  - Integrator: summarize the execution output and intermediate information to produce a final response
  - Parallel Batch Evolution: to increase throughput, generate a batch of responses in parallel. Each query has its own tool subset. Afterwards, to eliminate the redundant tools, we use:
    - Aggregator: to identify the tools in different subsets that have a similar function (*e.g.,*

search on Baidu and search on Google), re-group those tools by function. (LLM-based)
        - Merger: for each re-grouped set, re-write the tool to make it canonical.
- Experiment findings:
    - SOTA, even better than Gemini-3 Pro and GPT-5.
    - As time passes by, fewer new tools are created: good reusability of existing tools.
- My Thoughts:
    - Can the tool developer be a sub-agent-orchestra that can create more complicated tools, such as MCP servers?
    - After the Merger, the new tools will be added back to the repo. Here, another, I believe, similarity checker should be added to make sure the synthesized tool does have an obvious overlapping niche with what we currently have. (*From Manus's blog: a smaller tool set = more effective in tool use decision*)
    - Can decrease tons of GPU usage (cost) and time. Finetuning a task-specific agent by only interacting with it will be preferred by small companies and personal organizations that cannot afford heavy GPU usage (垂类agent系统训练成本也许会大幅度降低).
    - Add a memory system and a user feedback function. After the inference, the user can evaluate the inference quality. If good, add the task, used tools, and the whole chain of inference to the memory for quick reference in future tasks. *Perhaps can use this?*
- Work Cited
    - Paper
    - Project
    - Chat History